Understanding and Improving Human – AI Collaboration in Systems Engineering through Simulation

Ray Madachy
Naval Postgraduate School

INCOSE International Symposium 2026
Yokohama, Japan | June 13 – 18, 2026

Motivation

- Current LLM usage in systems engineering is largely ad hoc at individual and team levels.
- With no measurement, individuals and organizations lack visibility into generative AI time consumption and reliability.
- Need for a repeatable, measurable engineering process for generative AI.
- The Personal Generation Process (PGP) model treats human–AI interaction as a controllable engineering process with feedback rather than informal craft.

Wa わ

- This work strives for harmony, balance, and collaborative unity with AI by using simulation to better understand and improve the collaboration in a continuous process improvement feedback loop.
- The PGP model embodies this by treating human–AI interaction as a structured, measurable process in which the human practitioner remains both the operator and the reflective analyst of their own collaboration.
- Simulation of the process interaction can be used for learning, finding leverage points for improvement, and making strategic changes.
- Introspective simulation makes the dynamics of human–AI *wa* わ visible and quantifiable with controllable process parameters rather than invisible.
- PGP operationalizes awareness of one's role in the collaboration through empirical data collection, process modeling, and continuous improvement.

Process Simulation for Human–AI Collaboration

- A systems thinking approach to human–AI collaboration treats it as a process improvement problem, not just a tool evaluation.
- Simulation provides holistic process perspective of interrelated factors as a quantitative basis for deciding how to optimize AI value.
- Can model human–AI interaction as a controllable, hybrid discrete event process with continuous model elements.
- Can model feedback between user and AI, both within chats and across chats for learning and improvement using metrics.

PSP and TSP Methodologies (1/2)

- PGP is inspired by Watts Humphrey's Personal Software Process (PSP). He argued that system development problems must be addressed by treating development as a process that can be controlled, measured, and improved.
- The PSP emphasizes disciplined collection of individual-level data across the full development lifecycle for continuous improvement.
- PSP measures include time by development phase, size in lines of code, defect counts, defect density, and defect yields from reviews and inspections.
- Using an introspective approach, users develop analysis routines to evaluate their own process.
- The Team Software Process (TSP) implements this at the team level.

PSP and TSP Methodologies (2/2)

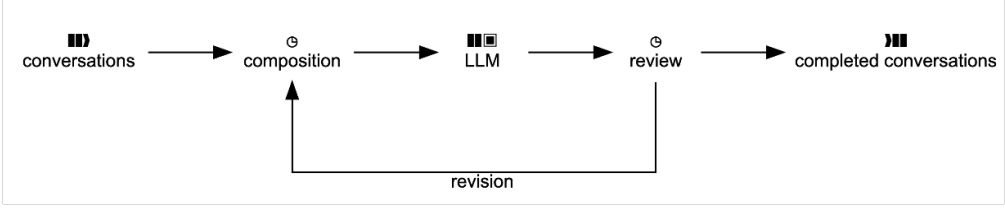
- Rigorous defect categorization enable understanding not only how many defects occurred, but what types and where they were injected and removed by what kinds of activities.
- This supports phase-based analysis of performance, planning accuracy, engineering effort, testing effectiveness, and rework burden.
- Data collection is not an end in itself, but the basis for estimation, process control, quality improvement, and professional discipline.
- PGP similarly frames generative AI usage as a measurable activity where the user is both the operator and the analyst.

Analogous Metrics for Human–AI Processes

- Metrics include prompt composition time, generation delay, revision time, review time, number of cycles, acceptance probability, total elapsed time, and output success rate.
- Additional introspection measures can include prompt size, degree of restructuring, and micro-edit frequency.
- AI defect categories of the generated product include factual error, omission, ambiguity, structural noncompliance, weak reasoning, formatting error, and insufficient task alignment.
- At the team level, analogous measures support a Team Generation Process (TGP) with shared prompt assets, reviewer roles, integration checkpoints, defect classification practices, and comparison across language models and workflows.

Generative AI Process Control Loop with Phases

- **Conversations:** Human user initiations of goal-oriented tasks consisting of prompts and responses.
- **Composition:** User formulates/revises prompts using domain knowledge.
- **LLM Invocation:** Stochastic processing time to generate model output.
- **Review:** User inspects LLM output against task objectives and standards.
- **Revision:** Iterating with a new composition if output is unsatisfactory by redefining scope or adding constraints.



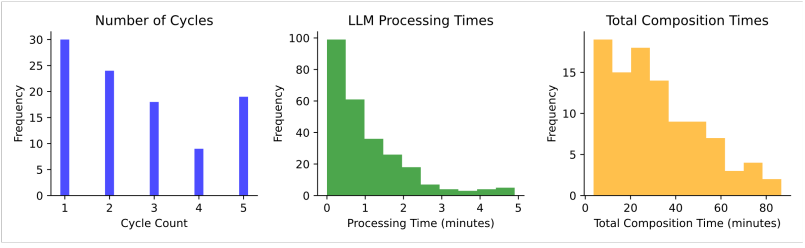
Simulation Implementation via se-lib

- Implemented in Python using the Systems Engineering Library (`se-lib`).
- Discrete event simulation with Monte Carlo analysis for stochastic behavior.
- Task durations modeled with initial probability distributions.
- Task sizes are calibrated to student level homework problems in systems modeling.

```
# Personal Generation Process Model

import selib as se
se.init_de_model()
se.add_source('conversations', entity_name="conversation",
              num_entities=1, connections={'composition': 1},
              interarrival_time=0)
se.add_delay(name='composition',
             delay_time='random.triangular(1,5,30)',
             connections={'LLM': 1})
se.add_server(name='LLM', connections={'review': 1},
              service_time='random.uniform(1,2)', capacity=1)
se.add_delay(name='review',
             delay_time='random.triangular(1,10,5)',
             connections={'completed prompts': .5, 'revision': .5})
se.add_delay(name='revision',
           delay_time = 'np.random.uniform(5, 10)',
           connections={'composition': 1},)
se.add_terminate('completed prompts')
se.draw_model_diagram()
```

Monte Carlo Simulation Results Example



Conclusions and Initial Findings

- Established a quantitative foundation for human–LLM interaction dynamics.
- Composition and review times dominate the total elapsed cycle time.
- LLM processing latency is negligible compared to human cognitive effort.
- Process variability is driven primarily by human decision-making.
- Baseline experiments show an overall success rate of approximately 85% within 5 iterations.
- Most tasks complete in few iterations, but ’long-tail’ cases present risk.

Future Work: Organizational Processes

- Single-user PGP is extensible to team-based workflows (analogous to TSP) to cover shared prompt libraries and defined reviewer/integrator roles in large projects.
- PGP loops can be embedded in larger systems engineering lifecycle models.
- Support decision making to estimate aggregate impact of AI on cost, schedule, and system reliability.
- Supports predictable and auditable integration of AI tools.

Future Work: Comparative LLM Performance

- Extend the model to parameterize for varying performance of different LLMs.
- Leverage current research on selecting, benchmarking, and deploying LLMs for systems engineering use.
- Represent differences in effectiveness, latency, consistency, review burden, model size, and token usage as explicit parameters.
- Examine how model choice influences acceptance probability, number of cycles, and total elapsed time.
- Consider practical constraints that vary by model, including memory footprint, compute demands, deployment feasibility, and local execution limits.
- Use comparative experiments to determine which model and workflow combinations are most effective and feasible for different systems engineering tasks.

Future Work: Interaction Modality

- Extend the model to represent alternative composition and editing modalities including typing versus voice dictation.
- Examine how input modality affects prompt entry speed, micro-editing effort, review burden, and total cycle time.
- Analyze the tradeoff between rapid idea capture and precise local control during iterative prompting.
- Measure when voice interaction improves throughput and when typing remains superior for exact wording and structured refinement.
- Use empirical logging of composition, revision, and review behavior to calibrate modality specific parameter distributions.

References

[1] Madachy RJ. *Software Process Dynamics*. Wiley-IEEE Press; 2008.

[2] Humphrey WS. *A Discipline for Software Engineering*. Addison-Wesley; 1995.

[3] Humphrey WS. *Introduction to the Team Software Process*. Addison-Wesley; 2000.

[4] se-lib development team. *se-lib: Systems Engineering Library (Python package)*. 2023–2026. Available from: https://pypi.org/project/se-lib/